Zero-knowledge proof

[TaSK]

November 2026

1 Innledning

Dette tekniske notatet gir en oversikt over en publisert artikkel [6, 7]. Fordi dette notatet beskriver en artikkel er her ingen nye resultater. Termer som "artikkelen", "kapittel 2", "figur 2", og tilsvarende refererer til [7]. Der det er nødvendig å referere til denne teksten bruker vi eksplisitt "i dette notatet".

Det gjøres ingen antagelser om kanaler som meldinger sendes på, utover ordning i tid (en melding ankommer ikke føær den er sendt). I særdeleshet antas det ikke at kanaler tilbyr konfidensialitet.

2 Bakgrunn

Tema er hvordan Alice kan overbevise Bob at hun kjenner en hemmelighet, uten å hverken fortelle noe av verdi til Bob (ut over at hun kjenner hemmeligheten), og heller ikke til Charlie (som ikke kjenner hemmeligheten) som lytter på samtalen. Det er en fordel om Charlie heller ikke kan konkludere at Alice kjenner hemmeligheten ut fra kommunikasjonen.

For eksempel: Gitt at Alice har konfidensielle data, for eksempel et dokument; la oss kalle dokumentet for D. Hun tilbyr ett eller annet til andre som også har dokumentet, for eksempel en form for login. Bob mener han har det, og ønkser å bevise det til Alice. Når følgende protokoll er ferdig er A overbevist om at B også har (tilgang til) D:

$B \longrightarrow A : \{$"`Kan jeg få adgang?`"$\}$

$A \longrightarrow B : \{N'_A\}$

$B \longrightarrow A : \{x \leftarrow H\{N'_A \,.\, D\}\}$

hvor notasjonen $\,.\,$ betyr sammensetting (*concatination*), N'_A "et tilstrekkelig stort tilfeldig tall" (la oss si $\approx 2^{256}$), og $H\{m\}$ et digitalit fingeravtrykk (*hashing*) av m. A beregner selv $y \leftarrow H\{N'_A \,.\, D\}$. Dersom $x{=}y$ kan A konkludere at B kjenner (eller har tilgang

til) D. Vi noterer i margen at denne enkle protokollen har mange svakheter og ikke tilfredsstiller moderne krav til å være sikker.

Fordi han lytter på all kommunikasjon kjenner Charlie nå N'_A og $H\{N'_A \,.\, D\}$. Det vil si at om han har stjålet et dokument D' kan han undersøke om $D'=D$ ved å beregne $r \leftarrow H\{N'_A \,.\, D'\}$ og $s \leftarrow H\{N'_A \,.\, D\}$. Dersom $r = s$ kan C konkludere at $D'=D$ med samme sikkerhet som A. Det vil at protokollen mellom A og B lakk (meta)informasjon til Charlie.

Evnen til å kunne verifisere om $D'=D$ (når D er ment å være hemmelig) er en betydelig fordel som kan gi næring til mange typer angrep. *Zero Knowledge* handler om å overbevise den andre om noe, uten å lekke informasjon. For å illustrere hva som menes, la Alice og Bob sitte på hver sin side av bordet, med en kortstokk mellom seg. Alice trekker et kort, og ser på det, og legger det med baksiden opp; kortet var rødt (hjerter eller ruter). Spørsmålet er: Hvordan kan Alice overbevise Bob om at hun trakk et rødt kort uten å vise ham det eller avsløre noe annet av verdi?

La Alice gå gjennom stokken og plukke ut de 26 sorte kortene (kløver og spar) og legge dem på bordet så Bob kan se dem. Under antagelsen om at kortstokken er "ekte" kan han konkludere uten mulighet for å gjøre feil at Alice trakk et rødt kort.

For at denne protokollen skal fungere, må Bob akseptere en rekke svært heftige antagelser. Den første handler om kortstokken; han må anta den har 52 kort, ikke inneholder en *joker*, har 13 kort av hver (kort)farge, samt den fysiske omgivelsen som må etableres (hva innbærer det å sitte på andre siden av bordet?).

Som vi skal se må det alltid gjøres antagelser; styrken i systemet avhenger av hva som legges til grunn. Kompleksitetsteori er verktøyet for å snakke om antagelser.

3 Matematisk utgangspunkt

Den matematiske konteksten i artikkelen er språk og kompleksitetsklasser; vi gir derfor en kort oppfriskning. Før vi kommer til artikkelens bidrag må vi introdusere to vekjente språk knyttet til grafer, og snakke litt om kryptering.

3.1 Språk

Alan Turing var den første til å representere (matematiske) spørsmål med symboler (tekst) [15]. Han representerte dem på datidens mest avanserte teknologi: Papirstrimmel. Det var først 25 år senere at det ble forstått at dersom et problem representeres som en kompakt streng (sekvens av symboler) så vil lengden av strengen være et mål på "størrelsen" av problemet [9]. For en introduksjon til anbefaler vi [2]; første kapittel går nøye gjennom representasjon av problemer.

I spørsmål om involverer tall er det åpenbart hvordan problemet skal representeres. For eksempel: Om spørsmålet er: "Er tallet et primtall?" så er instansen av problemet som skal besvares et tall. Tallet (problemet) kan være 323 (som vil si $(101000011)_2$). Vi har altså et spørsmål og et problem (i konktekst av spørsmålet).

Når spørsmålet omhandler en graf, for eksempel om grafen er forbundet, kan kan vi konvertere grafen til en matrise (hvor $M_{ij} = 0$ om det ikke er en kant mellom nodene i og j). Når en slik matrise skrives ut blir resultatet igjen en streng (av 0 og 1). Lengden på dette problemet er naturligvis produktet av dimensjonene.

Det finnes mer kompakte måter å gjøre det på, men i praksis spiller det ingen rolle. Konstant-leddet som en slik konvertering utgjør blir i praksis uten betydning.

Når vi bruker strenger til å representere problemer på denne måten kan vi betrakte strengene som et "ord"; hvert ord representerer ett (og bare ett) problem. Indirekte har vi nå også fått et mål på størrelse: Jo lenger streng jo større problem. Legg merke til at tallet 1234567890 er ti ganger så stort som tallet 123456789. Men som problem er 1234567890 bare 10% større (lengere). Det er fordi tall er en svært effektiv måte å representere problemer på.

Problemene til et gitt spørsmål er et (uendelig) sett av strenger. Spørsmålet deler dette settet to: De problemer som har svaret "ja" og de som ikke har svaret "ja". Kombinasjonen av et spørsmål, en representasjon av problemene som skal besvares, settet av alle problemer (ordene), og medlemmene av settet som har svaret "ja", det kaller vi *et språk*.

Noen velkjente språk er:

Primes er språket som inkluderer spørsmålet om $n \in \mathbb{N}$ er prim og settet med tall hvor svaret er "ja". Akkurat som ikke alle bokstavkombinasjoner tilhører språket norsk, tilhører ikke alle tall språket PRIMES

Factor Gitt to tall n og k, finnes det et tall $d \leq k$ som deler n, og

Clique er spørsmålet om en graf inneholder en komplett sub-graf (klikk) med en gitt størrslse, sammen med problemene som representerer grafer hvor svaret er "ja".

For å gjøre det enkelt setter vi som regel spørsmålet opp om "ja/nei"; "Finnes det..." heller enn "Hva er...".

3.2 Kompleksitesklasser

Det viser seg at språkene faller naturlig ned i et sett med klasser avhengig av hvor mye det krever å finne det; vi kaller dem kompleksitetsklasser. Klassene skiller seg fra hverandre ved noen sentrale egenskaper knyttet til arbeidsmengde.

Det essensielle er at alle spørsmål i et språk er "like". For eksempel er faktorisering av store tall som er produktet av kun to primfaktorer lik for alle tall av lik størrelse. Det

vil si at vi kan bygge en løsning rundt faktorisering uten på forhånd velge hvilke tall som skal inngå. Det gjør at hver instans er "frisk" og med samme egenskaper som alle andre.

3.2.1 Vekst

Når vi skal addere to tall, da legger vi først sammen siste siffer i begge tallene, deretter nest siste, og så videre. Det vil si at kompleksiteten vokser lineært med lengden på tallene (dobbelt så lange tall krever dobbelt så mange operasjoner). Vi ser utelukkende på tallenes lengde, ikke deres "størrelse". Kompleksiteten (arbeidsmengden) i addisjon vokser lineært med lengden på tallene som skal legges sammen.

Multiplikasjon krever at vi ganger hvert siffer i tallene med de andre, og summerer resultatene til slutt. Mest arbeid er det når tallene er like lange. Kompleksiteten i multiplikasjon er derfor N^2 hvor N her ikke er tallene men summen av lengden.

Addisjonen vi gjør til slutt blir fort neglisjerbar i forhold til selve multiplikasjonen fordi arbeidsmengden vokser mye raskere. En oppgave som (kun) vokser lineært med problemet kan neglisjeres når det inngår i en prosess som inkluderer noe som vokser polynomisk. Når vi maler med den brede pensel som kompleksitetsklasser er fokuserer vi kun på det leddet som vokser raskest.

Som regel kan vi bruke tid på å regne noe ut, eller pre-beregne for å tjene tid senere. Det vil si bytte vekst i tid mot vekst i rom. Det viser seg at problemer i en klasse ikke kan "bryte ut" av klassen fordi vekst i rom "koster" like mye som vekst i tid. For eksempel kan vi løse multiplikasjon i fast tid ved å ha en ferdig multiplikasjonstabell. Men den har jo størrelse N^2 og i praksis er derfor ingenting spart. Derfor, når kompleksiteten er, for eksempel, $\mathcal{O}(N^2)$ er det enten i tid eller i rom.

For en introduksjon til aymptotisk vekst anbefaler vi [8] eller, naturligvis [11].

3.2.2 Klassen P

Et språk det krever $\mathcal{O}(N^2)$ beregninger å bestemme er klart lettere å håndtere enn ett som krever $\mathcal{O}(N^3)$. Men som alltid er alt relativt: I forhold til $\mathcal{O}(2^N)$ er forskjellen mellom $\mathcal{O}(N^2)$ og $\mathcal{O}(N^3)$ så liten at de er "like".

De språkene som kan konstrueres i tid og rom som ikke vokser mer enn polynomisk relativ til lengden på spørsmålet, de sies å tilhøre klassen P.

For eksempel PARTALL: Er tallet $n \in \mathbb{N}$ partall? Algoritmen for å bestemme om et tall er i PARTALL er å se om siste bit er 0 eller 1. Det tar fast tid (uavhengig av lengen).

STI som gitt en graf G og to noder a, b spør om det er en sti mellom dem. Hverken bredde-først eller dybde-først vokser raskene enn antall noder den skal besøke. Det vil si at veksten er polynomisk når størrelsen på G er målet. Språket STI er derfor i P.

Det er lett å finne ut om n er prim (prøve å dividere med alle tall $< \sqrt{n}$) om enn tidkrevende om n er stor. Som problem i et språk måler vi N som lengden. Et tall som er dobbelt så *langt* som et annet, for eksempel 20 mot 10 siffer, det er svært mye større. Denne algoritmen gir derfor eksponensiell vekst i antall operasjoner (målt mot N som er lengden av n). Heldigvsi finnes det mer effektive algoritmer; det beste vi kjenner er $\mathcal{O}(\log^6 n)$ [1]. Det er avgjørende her å se at å avgjøre om et tall er prim eller ei er noe annet og mye lettere enn å faktisk finne primfaktorene. Språket PRIMES er derfor i P.

3.2.3 BPP (Bounded-error Probabilistic Polynomial time)

For at beskrivelsen skal være korrekt nevner vi at fordi det ikke er bevist at *psaudorandomness* i det generelle tilfellet kan erstatte ekte tilfeldighet, så er $\mathsf{P} \subset \mathsf{BPP}$. Men som alle andre antar vi at så er tilfelle, og at $\mathsf{P} = \mathsf{BPP}$.

3.2.4 Klassen NP

En annen klasse, som inkluderer settet P, er NP. Språk i denne klassen er de hvor det er mulig å verifisere et bevis[1] på at en instans har "ja"-svar i polynomisk tid. At det finnes et bevis, som, igjen, er noe annet enn å finne beviset, er en sentral egenskap knyttet til språk

For å se at P er innholdt i NP er det tilstrekkelig å notere at elementene i P kan konstrueres i polynomisk tid. Det vil si det ikke er behov for noe bevis i det hele tatt; man kan bare regne ut løsningen.

Et språk som ofte brukes som eksempel er SAMMENSATT: Gitt et heltall n, er det sammensatt (ikke prim)? Et bevis for at n er sammendatt (et "ja"-svar) er $n = xy$.

på x og y (som naturligvis øker med lengden på n), men kun polynomisk. Altså er SAMMENSATT i NP. Kjernen er at det som skal gjøres er ikke å finne ut om et tall er sammensatt, men undersøke beviset er gyldig.

PRIMES er også i NP fordi det finnes et bevis som kan verifiseres i polynomisk tid[2]. Dersom n er prim da vil det finnes en $a^{n-1} = 1(= n)$ og for hver primfaktor q av $n - 1$ vil $a^{(n-1)/q} \neq 1(= n)$. Et bevis for at n er prim vil være a sammen med faktoriseringen av $n - 1$ (og tilsvarende bevis for at de faktorene er prim).

Minner igjen om at å finne ut om n er prim er ett problem mens avgjøre om beviset på at n er prim er gyldig, er et annet. Kravet for at et problem er i NP er altså at det finnes et bevis (og ikke nødvendigvis at beviset er "lett" å finne).

[1] På engelsk har *proof* og *evidence* forskjellig betydning. Når vi snakker om *bevis* her mener vi *evidence*.

[2] PRIMES ble opprinnelig (1975) plassert i NP. Senere har det vist seg å være i P, altså "enklere" enn man opprinnelig trodde.

3.2.5 Klassen NPC

Noen problemer som har enkel bevis, er svært vanskelig å løse. For eksempel

- TRAVELLING SALESMAN: Gitt en graf med vektede kanter, finnes det en rute med lengde $< k$ som besøker alle noder (byer)? Beviset er ruten, som er lett å verifisere har lengde mindre enn k;
- SEKKEPROBLEMET: Gitt et sett av gjenstander med volum og verdi, finnes det et subsett av gjenstandene som til sammen har verdi $> k$ og som til sammen ikke har større volum en sekken?, og
- FARGELEGGE EN GRAF: Gitt en graf hvor kantene ikke har retning, og et antall farger, finnes det en måte fargelegge nodene slik at ingen node har kant til en annen node med samme farge?

Disse, og mange andre (se [4] for mer enn 300 eksempler) er "like", i betydningen at de har kompliksitet som minst er $\mathcal{O}(2^N)$.

Problemene i NPC har vært studert av informatikere og matematikere i flere tiår. Det er ingen tegn på at noen av dem har enkle løsninger.

Definisjon 3.1 (Umulig)**.** Noe er *umulig* dersom det er like vanskelig, eller vanskeligere, enn problemene i NPC.

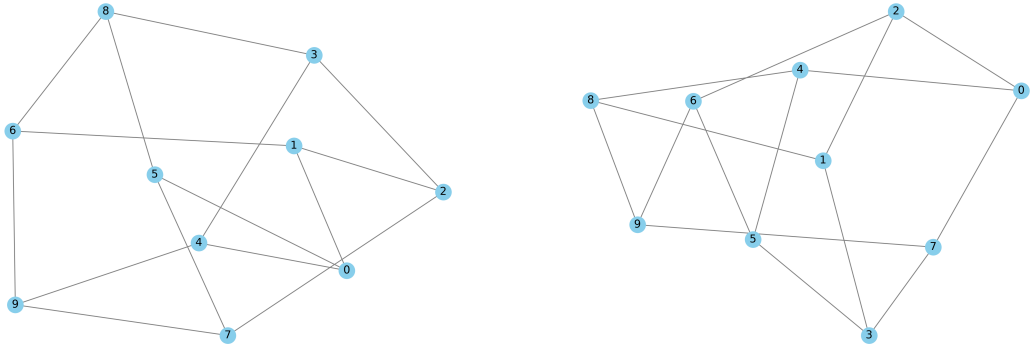
Denne definisjonen er naturligvis en svært uformell måte å uttrykke hvor mange beregninger som i det generelle tilfelle er nødvendig for å avgjøre om en streng $x \in L$, hvor $L \in$ NPC.

Det må understrekes at det er forventningen (les: sannsynligheten) som er hindringen. For eksempel: Gitt et 60 sifret tall tall som er produktet av to 30-sifret primtall, da er det mulig å være heldig å gjette de to faktorene på første forsøk. Sannsynligheten for det er svært liten, og derfor starter vi ikke engang på oppgaven. Noe er derfor *umulig* dersom den forventede arbeidsmengden PER SE gjør at man avstår.

Her noterer vi med en viss interesse at hverken RSA (basert på FACTOR eller ECDSA (basert på DLG) er *umulig* å knekke. Vanskelig, men ikke *umulig*.

3.3 Eksempler

Fordi klassen NPC er sentral i artikkelen skal vi gå gjennom to eksempler; begge er grafer. Det ene i NP men utenfor BPP det andre i NPC.



(a) Grafen G_1 (b) Grafen G_2

Figur 1: To grafer som er isomorfe. Hele poenget er at det ikke er lett å se. Mappingen her er 0:1, 1:2, 2:0, 3:4, 4:8, 5:5, 6:9, 7:6, 8:3, 9:7.

3.3.1 Isomorfe grafer

To grafer $G_1 = (V_1, E_1)$ og $G_2 = (V_2, E_2)$ er *isomorfe* (vi skriver $G_1 \cong G_2$) dersom, og bare dersom, det finnes en *bijeksjon*

$$f : V_1 \to V_2$$

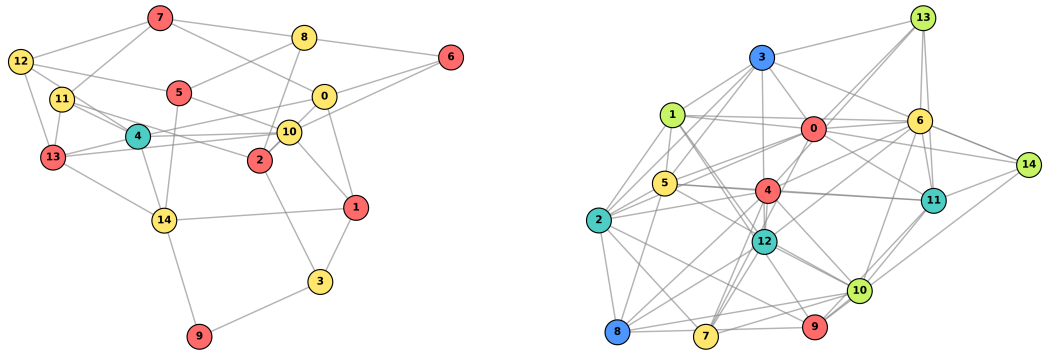
slik at dersom to vilkårlige noder $u, v \in V_1$ er forbundet med en kant i G_1 da er deres avbildinger $f(u)$ og $f(v)$ forbudet med en kant i G_2. I praksis betyr det at grafere har lik struktur, selv om navnet på noder og kanten er forskjellig.

Å generere en graf er lett, og det er enkelt å generere en isomorf kopi av en graf. Det er i praksis "bare" å flytte rundt på det som identifiserer nodene og bygge opp mappingen mens man gir nodene nytt navn. Overgangen mellom to isomorfe grafer G_1 til G_2 er altså en liste som viser hvordan identifikatorene på nodene er knyttet til hverandre. Listen er naturligvis like lang som det er noder i grafen. En slik mapping kan brukes begge veier, og i flere ledd; én graf brukes til å generere en ny, som brukes…Vi skal komme tilbake til denne egenskapen litt senere.

Selve mappingen er en liste med navn (på noder, se Figur 1). Den forteller ingenting om grafenes struktur (som er det isomorfi handler om). Særlig: Gitt en graf og en mapping kan man (kun) lære at det finnes en isomorf graf, men verdien av den kunnskapen er null ettersom det for alle grafer kan lages en

Det er ikke er lett å avgjøre om to grafer er isomorfe, selv om det ikke er *umulig*. Så vidt man vet er språket GRAFISOMORFISME ikke i P. Gitt en mapping fra én graf til en annen er det trivielt å verifisere at to grafer er isomorfe: GRAFISOMORFE er derfor klart i NP.

3.3.2 Fargelegge en graf



(a) En graf riktig farvelagt

(b) Kan ikke fargelegges med 3 farger

Figur 2: To grafer med tre og fire farger, respektive

Gitt en graf, kan man fargelegge alle nodene med ett gitt sett $\phi = \{1, 2, 3, \ldots, n\}$ farger, slik at ingen noder med lik farge er forbundet med en kant? For å gjøre oppsettet så enkelt som mulig velger vi $n = 3$. Det er da åpenbart at en graf hvor det finnes noder med mer enn tre kanter, så er svaret nei.

La $\mathcal{L}$ være språket om holder alle grafer hvor det er mulig å farvelegge nodene med $n = 3$ farger uten at noen noder med lik farge er knyttet sammen med en kant; språket kalles 3-COLOUR.

Det har vært kjent i mer enn 50 år at å avgjøre om en graf er med i 3-COLOUR er i NPC [4, 10]. Det vil si at å svare på om en gitt graf er i språket, er *umulig*. Videre, ethvert spørsmål som på en eller annen måte kan gjøres om (konverteres) til 3-COLOUR er *umulig*; dette skal vi bruke senere.

3.4 Språkene i klassen er "like"

Det er svært nyttig å rubrisere problemer i kompleksitetsklasser. Vi kan fokusere på klassene heller enn på nyanser i hvert problem. Klassen NPC ble kjent allerede i 1971 [3], og siden 1973 har man visst at problemene kan *reduseres* til hverandre [10]. Det er altså en egenskap ved problemene i NPC at de er "like vanskelige", og svært mye "vanskeligere" enn "vanlige" algoritmer.

I forhold den massive innsatsen som må til for å svare på et problem fra NPC er det bagateller som skal til for å konvertere det til et annet. I dag brukes det som bevis på at et problem er i NPC dersom det er mulig å "skirve det om" til ett av de andre problemene i settet.

Vi skal benytte denne egenskapen senere.

3.5 Kryptering

Før vi kan gå videre trenger vi å diskutere hvilke antagelser kryptering krever. Det vanlige er å helt enkelt anta at FACTOR og/eller DLG[3] ikke er i BPP. Et viktig tema i dette notatet er å få grep om antagelser, og da er kryptering viktig.

Det lar seg ikke gjøre å bevise at kryptering fungerer. Det vi kan gjøre er å vise at problemer er i samme kompleksitetsklasse. Så får vi bare håpe at implementasjonen ikke svekker de teoretiske antagelsene selv om nær 100% av alle svakheter nettopp er knyttet til programvare heller enn teori.

Shannon [13] skrev for et par generasjoner siden at engangs-krypto[4] under antagelsen om "ekte" tilfeldighet er sikkert; for å si det forsiktig er diskusjonen om "ekte" tilfeldighet ganske omfattende og vi lar det ligge. Det er tilstrekkelig å vite at kryptering kan gjøres helt (!) vanntett om man ønsker[5].

I praksis må vi anta noe mer anvendelig. Den mest benyttede antagelsen er fra vår egen levetid og er at krypteringen er sikker i møte med enhver som "kun" er utstyrt med maskiner som kan avgjøre om et spørsmål i et språk som er med i BPP [5]. En annen måte å si det på, er at med mindre en kjent (og akseptert) antagelser–for eksempel $\mathsf{P} \neq \mathsf{NP}$–ikke lenger holder, så holder krypteringen (igjen, med forbehold om feil i programvare).

Om sikkerheten ikke er så nøye, da kan man bruke en PKI for å knytte entiteter som deltagere og kanaler (krypteringsnøkler) sammen [12]. Fordi offentlig-nøkkel kryptografi hviler på alt for mange antagelser legger vi slike løsninger bort uten noe om-og-men. Det eneste kloke er å bruke delt-nøkkel kryptering til å sikre informasjon som brukes til å formidle kunnskap om hemmeligheter.

Som kjent har kvantedatamaskineren annen beregningsmodell enn det definisjonen av P og NP bygger på. Det ser derfor ut til at de kan svare på spørsmål i språk utenfor BPP i polynomisk tid. Her ligger årsaken til at kvantemaskiner og kryptering ofte okkuperer plass i samme setning. Vi sover likevel godt om natten vel vitende om at kryptografi som kan motstå kvante-maskiner er velkjente. Rykter forteller at [14] er en god introduksjon.

4 Problemstilling

Eksempelet med kortstokken vi seksjon 2 illustrerte essensen i en ZERO KNOWLEDGE-PROTOCOL. For bruk mellom datamaskiner hvor det ikke er aktuelt å sette seg ned ved bordet, og i særdeleshet for å formidle hemmeligheter, er det nødvendig med en noe mer komplisert omgivelse

[3] Discrete Logarithm Problem

[4] På nynorsk *one-time pad*.

[5] Ordet *umulig* er gitt en annen betydning og kan derfor ikke brukes her.

På et strukturelt nivå ønsker vi at Charlie, som lytter på kommunikasjonen mellom Alice og Bob, er i samme beregningsmessige posisjon etter å ha lytteat på protokollen mellom Alice og Bob som han var før protokollen startet. Altså at kompleksiteten i å finne x ikke endrer seg gjennom å lytte mens Bob på sin side blir overbevist om at Alice kjenner x.

Vi ønsker å finne et språk $\mathcal{L}$ som er slik at Alice kan overbevise Bob om at hun kjenner $x \in \mathcal{L}$ uten å formidle noe av verdi om x. Konkret:

- Dersom $x \in \mathcal{L}$ da kan Alice overbevise Bob om det med en sannsynlighet $P > 1 - |x|^c$ for enhver $c > 0$, og
- Dersom $x \notin \mathcal{L}$ da vil Bob kunne avvise at $x \in \mathcal{L}$ med en sannsynlighet $P > 1 - |x|^c$ for enhver $c > 0$.

Vi kan senke sannsynligheten for at noen gjetter våre krypteringsnøkler ved å velge dem lange; vi husker at "senke" ikke medfører "sette til 0". På sammen måte kan Bob senke sannsynligheten for at Alice lurer ham til et nivå han kan leve med. Motto er "Tenk på et lite (nok) tall".

4.1 Protokoll basert på isomorfi

Vi skal nå beskrive hvordan Alice kan overbevise Bob om at hun kjenner en hemmelighet basert på mappingen mellom to isomorfe grafer, uten å avsløre noe om grafene. Det vil si: en Zero-Knowledge Protocol. Protokollens evne til å senke sannsynligheten henger på at Grafisomorfi ikke er i P.

Grafene G_1 og G_2 er offentlige, formodentlig produsert av B. Han har først generert G_1 og fra den laget en kopi, med andre identifikatorer på nodene. Fordi han har laget de to grafene, kjenner han mappingen mellom nodene.

A kjenner også mappingen, og ønsker å overbevise B om at så er tilfelle. Protokollen har tre meldinger kjøres til A er fornøyd; vi kommer tilbake til hva det konkret betyr.

$M_1 \quad B \longrightarrow A : H$

$M_2 \quad B \longrightarrow A : R_\alpha$

$M_3 \quad A \longrightarrow B : M$

Meldingene og beregningene er som følger:

1. A generer en isomorf kopi H av G_1. Mappingen mellom G_1 og H kaller vi m_1 og mappingen mellom G_2 og H kaller vi M_2. A sender H til B. C ser kun en graf han mistenker er isomorf med G_1 men kan i praksis ikke verifisere at det er tilfelle (fordi Grafisomorfi ikke er i $\mathcal{BPP}$);
2. Etter å ha mottatt H, genererer B et tilfeldig tall $\alpha \in \{1, 2\}$ og sender til A. Vi noterer at dette steget åpner spørsmålet om protokollen er i P eller BPP – et spørs-

mål vi naturligvis lar ligge;

3. Dersom $\alpha = 1$ da sender A M_1 til B. Dersom $\alpha = 2$ sendes M_2. I begge tilfellene ser C kune en liste med verdier som er like lang som det er noder i grafen.

Alice har $p = 1/2$ sannsynlighet for å svare rett. Det vil si at for hver runde Alice svarer riktig synker sannsynligheten for at Alice bare gjetter til halvparten. Bob kan be Alice fortsette (kjøre runde etter runde) til han er overbevist (sannsynligheten blir tilfedsstillende lav).

Her er vi ved en ny antagelse: Akkurat som det er like usikkert at GRAFISOMORFI er i BPP som at FAKTORISERING er det–alle som bruker RSA i sin PKI (for eksempel PIT) antar at P $\neq$ BPP–en antagelse at $p = 2^{-64}$ (eller hva nå Bob velger) er tilstrekkelig til å kategorisere det som "sikkert".

Vi bruker vår intuisjon, og konkluderer med at konseptet ZERO-KNOWLEDE PROTOCOL faktisk lar seg realisere, og vil fungere under de rette antagelsene. Imidlertid er det ikke *umulig* å bryte sikkerheten, for det underliggende problemet (GRAFISOMORFI) ikke er i NPC.

Et spørsmål for *connoisseurs* er om resultatet (for Bob) blir det sammen om rundene kjører i parallell; det åpner svært utfordrende analytiske arenaer man kan danse på, men vi går ikke nærmere inn på. det her.

4.2 Protokoll basert på 3-colour

Igjen er oppsettet at Bob ønsker å overbeivse Alice om at han kjenner en hemmelighet, uten å lekke informasjon. Her er det fristende å foreslå "Han kan bare sende hemmeligheten kryptert til Alice for å bevise det", men målet er å heller ikke lekke informasjon til ham; det ville være katastrofe å sende hemmeligheten til feril mottaker; igjen, de som drømmer om at PKI løser utfordringene må sette seg inn i antagelsene.

Denne gangen er hemmeligheten pakket inn somfargeleggingen av nodene i en graf; vi velger 3 farger uten tap av det generelle.

M_1 $\quad B \longrightarrow A :$

M_2 $\quad B \longrightarrow A : R_\alpha$

M_3 $\quad A \longrightarrow B : M$

Bob har publisert en graf $G = (VE)$ som han vet kan fargelegges; fargelkeggingen er hemmeligheten. Alice ønsker å overbevise Bob om at hun kjenner hemmeligheten (fargeleggingen). Fargeleggingen Bob har klaget kalles $\phi : V \in \{1, 2, 3\}$ og viser hvilke noder som har hvilken farge.

Protokollen har tre meldinger, og gjentas til Bob fornøyd; mer om hva det innebærer nedenfor.

1. A genrerer en kopi av G og randomiserer nodenavnene. Det gir en annen (men strukturelt lik) fargelegging av nodene og formelt er $\pi \in_R Sym(\{1,2,3\})$ (som betyr at av settet av alle permutasjoner av settet av farger (her 3) velges én). A genererer tilfeldige krypteringsnøkler r_v og beregner $R_v = krypt(\pi(\phi(v)), r_v)$ for alle $v \in V$. A sender de kryptere meldingene $R_1, R_2, ...R_n$ til B;

2. B velger en tilfeldig kant $e \in_R E$ og sender den til A. Det kan vi tenke på som at B ver A vise hun kjenner fargelegggingen av de nodene som kanten knytter sammen. Husk at G er offentlig slik at navnene på kantene er kjent, og

3. Dersom $e = (u, v) \in E$ beviser A at hun kjenner fargene i disse to nodene ved å sende til B $(\pi(\phi(u), r_u)$ og $(\pi(\phi(v)), r_v)$ (innholdet i to i serien av krypterte meldinger sendt i 0).

Nå vil B verifisere at $R_u = krypt(\pi(\phi(u)), r_u)$, $R_v = krypt(\pi(\phi(v)), r_v)$, $\pi(\phi(u)) \neq \pi(\phi(v))$, og at $\pi\phi(u)), \pi((\phi(v)) \in 1, 2, 3$.

Når A skal svare på Bs spørsmål om fargene på to noder, hun $p = 1/3$ sannsynlighet for å gjette rett på hver. Hun har derfor bare $p = 1/9$ for å gjette rett på begge. For hver runde øker Bs tillit til at A kjenner hemmeligheten tilsvarende. Fordi A genererer en ny permutasjon av G for hver runde lekker ingen informasjon om farveleggingnen. Det betyr at B kan be om så mange runder som ønskelig for å få tilfredsstillende sikkerhet.

5 Utvidelser

Artikkelen viser at 3-COLOUR kan brukes til å realisere en ZERO-KNOWLEDLE PROTOCOL. Fordi de kan reduseres til hverandre, kan alle de andre problemene i NPC kan brukes til å bygge en slik protokoll.

6 Notasjon

Referanser

[1] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. PRIMES is in $\mathcal{P}$. *Annals of Mathematics*, 160:781–793, 2004.

[2] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 1st edition, 2009.

[3] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the third annual ACM symposium on Theory of computing*, STOC '71, pages 151–158. ACM, 1971.

[4] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of $\mathcal{NP}$-Completeness*. W. H. Freeman and Company, San Francisco, 1979.

[5] Shafi Goldwasser and Silvio Micali. Probabilistic encryption. *Journal of Computer and System Sciences*, 28(2):270–299, 1984.

[6] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof-systems. In *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing*, STOC '85, pages 291–304. ACM, 1985.

[7] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.

[8] Ronald L. Graham, Donald E. Knuth, and Oren Patashnik. *Concrete Mathematics: A Foundation for Computer Science*. Addison-Wesley, Reading, MA, 2nd edition, 1994.

[9] Juris Hartmanis and Richard E. Stearns. On the computational complexity of algorithms. *Transactions of the American Mathematical Society*, 117:285–306, 1965.

[10] Richard M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pages 85–103. Springer, 1972.

[11] Donald E. Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley Professional, Boston, 3rd edition, 1997.

[12] Butler W. Lampson, Martín Abadi, Michael Burrows, and Edward Wobber. Authentication in distributed systems: Theory and practice. *ACM Transactions on Computer Systems*, 10(4):265–310, 1992.

[13] Claude E. Shannon. Communication theory of secrecy systems. *Bell System Technical Journal*, 28(4):656–715, 1949.

[14] Douglas R. Stinson. *A Primer on Post-Quantum Cryptography*. Chapman & Hall/CRC Cryptography and Network Security Series. CRC Press, 2026.

[15] Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1):230–265, 1937.